CS4433 – Moderated Learning Management System (LMS)

BUILD GUIDE

GitHub Repository:
https://github.com/Kevin-droid408/CS4433-Moderated-Learning-Management-System

1. Introduction

This build guide explains how to configure, compile and run the Moderated Learning Management System (LMS) on a Windows computer using Visual Studio Code, MSYS2, GNU C++ (g++), CMake and SQLite.

The guide is intended for developers or users who want to build the application from its source code.

2. Development Environment

The project uses the following tools and technologies:

Tool / Technology	Purpose
C++17	Main programming language
Visual Studio Code	Development environment
GNU C++ (g++)	C++ compiler
MSYS2 UCRT64	C++ development toolchain
CMake	Build configuration and management
MinGW Make	Build tool
SQLite	Database management
libsodium	Password hashing and verification

3. System Requirements

Before building the project, ensure that the following software and libraries are installed:

- Windows operating system.
- Visual Studio Code.
- MSYS2 with the UCRT64 environment.
- GNU C++ compiler (g++).

- CMake version 3.20 or later.
- MinGW Make.
- SQLite development library.
- libsodium development library.

Ensure that the MSYS2 UCRT64 binary directory is available in the system PATH:

C:\msys64\ucrt64\bin

4. Obtaining the Project

Clone the project repository from GitHub.

Open the Visual Studio Code terminal and execute:

git clone https://github.com/Kevin-droid408/CS4433-Moderated-Learning-Management-System.git

Navigate into the project directory:

cd CS4433-Moderated-Learning-Management-System

Alternatively, download the repository from GitHub and open the extracted project folder in Visual Studio Code.

5. Verifying the Development Tools

Before configuring the project, verify that the required build tools are available.

Step 1: Verify CMake

cmake --version

CMake should display its installed version.

Step 2: Verify the C++ compiler

g++ --version

The command should display the installed GNU C++ compiler version.

Step 3: Verify MinGW Make

mingw32-make --version

The command should display the installed MinGW Make version.

If any command is not recognised, check that the relevant software is installed and that its directory has been added to the system PATH.

6. Project Configuration

The project uses the CMakeLists.txt file to define its build configuration.

This file specifies:

- The minimum required CMake version.
- The C++17 language standard.
- Application source files.
- Header file directories.
- External library dependencies.
- Executable targets.

The application requires SQLite and libsodium to compile and link successfully.

7. Building the Project

Open the project folder in Visual Studio Code and select **Terminal → New Terminal**.

Ensure that the terminal is positioned in the project root directory.

Step 1: Configure the project

Execute the following command:

```
cmake -S . -B build -G "MinGW Makefiles" -
DCMAKE_MAKE_PROGRAM="C:/msys64/ucrt64/bin/mingw32-make.exe"
```

This command:

- Uses the current directory as the source directory.
- Creates a build directory.
- Selects MinGW Makefiles as the build system.
- Specifies the location of the MinGW Make executable.

CMake will configure the project and locate the required dependencies.

Step 2: Compile the application

After successful configuration, execute:

cmake --build build

CMake will compile the project's source files and link the required libraries.

If compilation completes successfully, the application executable will be generated in the build directory.

8. Running the Application

After building the project, ensure that the terminal is still in the project root directory.

Run the main application using:

.\build\ModeratedLMS.exe

The application should start and display the login interface.

Users can log in using their registered account credentials.

9. Initial Administrator Setup

If the database does not already contain an Administrator account, use the provided account creation utility if its target was successfully built.

Execute:

.\build\CreateAdministrator.exe

Follow the prompts to create the initial Administrator account.

The Administrator can then log in to manage users, departments and courses.

10. Build Verification

The build process is considered successful when:

1. CMake configures the project without errors.
2. The C++ source files compile successfully.
3. SQLite and libsodium dependencies are located and linked successfully.
4. The required executable targets are generated.
5. The main application launches successfully.
6. The login interface is displayed.

11. Troubleshooting

CMake is not recognised

If PowerShell reports that cmake is not recognised, verify that CMake is installed and added to the system PATH.

G++ is not recognised

Check that the MSYS2 UCRT64 compiler is installed and that the following directory is included in PATH:

C:\msys64\ucrt64\bin

MinGW Make is not found

Verify that mingw32-make.exe exists in:

C:\msys64\ucrt64\bin

You can also specify its location using the CMake configuration command in Section 7.

SQLite or libsodium cannot be found

Ensure that the development libraries are installed in the MSYS2 UCRT64 environment. Confirm that CMake can locate their headers and libraries.

Compilation fails

Read the error messages displayed in the terminal. Check the affected source file, header files, dependencies and CMake configuration before rebuilding.